Practice Problems for the Final Exam

Identifying Errors

There are one to two problems (build errors or undefined behavior) in each of the following code snippets. Identify the problem and give an explanation for what is causing the problem.

Problem 1

```
int main () {
    int x = 4;
    int& y = x++;
    int& z = ++x;
}
```

Answer

Post-fix increment, `x++` will return by value, meaning that the return value is an r-value. A non-const l–value reference, `int& y` cannot be initialized by an r-value, so this code will have a **build error**.

Problem 2

```
int* new_array1(int&& k) {
    int p[3];
    p[0] = k;
    p[1] = k + 1;
    p[2] = k + 2;
    return p;
}
int* new_array2(int&& k) {
    int* p = new int[3];
    p[0] = k;
    p[1] = 2 * k;
    p[2] = 3 * k;
    return p;
}
int main() {
    int* p1 = new_array1(4);
    int* p2 = new_array2(5);
    cout << *p1 << " ";
    cout << *p2 << " ";
    delete[] p1;
    delete[] p2;
}
```

Answer

There are two issues with this code, this first can be seen in the function body of `new_array1`. In the code, a c-style array is created with `int p[3]`. This creates data on *the stack*, but more importantly, it is a local variable to the scope of `new_array1`. This means that by returning a pointer to that data, the function is returning a pointer to a local variable. Once the function's call is over, that local variable may be deallocated and used for something else, so using the pointer from that function call, `p1` will give **undefined behavior**.

The second issue with the code is that `p1` is a pointer to a stack variable. The end of the code calls `delete` on `p1`. Calling `delete` on a stack variable gives **undefined behavior**.

Problem 3

```
class C {
    public:
    int c;
    explicit C(int a) : c(a) {}
    operator double() {return c;}
};
int operator+(C& a, int y) {return a.c + 2*y;}
double operator+(C a, double y) {return a.c + 3*y;}
int main () {
    C x(2);
    x.c = 3;
    cout << x + 1.0 << endl;
    cout << C(3) + 2.0 << endl;
    cout << 2.0 + C(3) << endl;
    cout << C(1) + 1 << endl;
}
```

Answer

Here there are 3 `operator+` calls being invoked. The first will call `double operator+(C a, double y)` because the overload resolution will pick the cadidate with the least casting. In this case, `x` is already type `C` and `1.0` is already type `double`. The second `+` will also call `double operator+(C a, double y)` for the same reason, no casting needs to happen.

The third `+` cannot directly call either of the user-defined `operator+`s because the numeric type is first. So the overload resolution must decide on how to implicitly cast the operands in the best way. Because we have the `explicit` keyword in front of `C(int a)`, the compiler cannot create a `C` type from a int implicitly. Thus, the only option is to implicitly convert `C(3)` to a double, which is allowed because we defined the casting operator `double()` (with the `explicit` keyword).

The final `+` gives us problems. We cannot use `int operator+(C& a, int y)` because `C(1)` is an r-value, and thus cannot bind to the l-value reference in the function signature. So the overload resolution must resort to casting. The two options are to cast `1` to a double and call `double operator(C a, double y)` or to cast `C(1)` to a double and add a double plus an int. Neither one of these is seen as better by the compiler so the operator call is ambiguous.

Problem 4

```
const int& sq(int a) {
    return a*a;
}
int main() {
    int x = 4;
    const int& x_squared = sq(4);
    const int& x_to_the_4th = sq(sq(4));
    int x_cubed = x * sq(x);
    cout << x_squared << endl;
    cout << x_cubed << endl;
    cout << x_to_the_4th << endl;
}
```

Answer

We can immediately see a problem with this `sq` function because it is returning by reference, but the argument is not a reference. Whenever this happens (unless there is some weirdness involved with static or global variables), this means that a reference to a local function variable is being returned. We will get undefined behavior from `const int& x_sqared = sq(4);` because the reference `x_squared` is referencing a local variable that will be freed up after the function call is over. Using `x_squared` later in the problem gives us undefined behavior (in my case, a segmentation fault).

Note that even the line `int x_cubed = x * sq(x);` will give undefined behavior, because the reference to the temporary object becomes a dangling reference *immediately*. So even if we were to write something like `int x_squared = sq(4);`, we would *still* get undefined behavior as the initialization of `x_squared` would try to grab the data at the memory location of a local function variable (which has already been freed as the function call ended).

Predicting Output

Each of the following examples builds without undefined behavior. Predict the output in each of the following code examples.

Problem 1

```
class D {
public :
    int c;
    D(int _c) : c(_c) {cout << "c1 ";}
    D (const D& other) : c(other.c) {cout << "c2 ";}
    D (D&& other) : c(other.c) {cout << "c3 ";}
    ~D() {cout << "d ";}
    int* foo() {
        cout << "p1 ";
        return new int(c);
    }
    D operator+(D other){
        cout << "+ " ;
        return c+other.c;
    }
};
int main () {
    D x(3);
    D y(x);
    x + 3;
    D z(move(x));
    int* p = y.foo();
    y.c = 4;
    cout << *p << " " << y.c << " ";
    delete p;
}
```

Answer

First, `D x(3);` will invoke the constructor `D(int _c)` and print `c1`. Then `D y(x)` will invoke the copy constructor, `D(const D& other)` and print `c2`. Then `x + 3;` will implicitly cast `3` into a `D` type object and use the `operator+(D other)`. This will print `+`. Next `D z(move(x))` will invoke the move constructor, printing `c3`. Then `int* p = y.foo();` will invoke the member function `int* foo()` and print `p1`. The program then assigns `4` to `y.c` and prints `*p` (3) and `y.c` (4). So the total output will be

```
c1 c2 + c3 p1 3 4
```

Problem 2

```
int foo(string& x) {return 1;}
int foo (string&& x) {return 2;}
int foo (const char* x) {return 3;}
int foo (char* x) {x[0] = 'b'; return 4;}
```

```
int main () {
    char* p = new char('a');
    cout << foo(p) << " ";
    cout << foo("ljksd") << " ";
    cout << foo(string("abc")) << " ";
    string x = "lksd";
    cout << foo(move(x)) << " ";
    cout << foo(x) << " ";
    delete p;
}
```

Answer

The first function call `foo(p)` will invoke `foo (char* x)` returning `4`. The second call will invoke `foo (const char* x)` (recall that a string literal is a `const char[]`, which will return `3`. The call `foo(string("abc"))` will invoke `foo(string&& x)` returning `2`. The call `foo(move(x))` will also invoke `foo(string&& x)` because `move(x)` casts `x` to an r-value reference. Finally, `foo(x)` will invoke `foo(string& x)` returning `1`. So the output is

```
4 3 2 2 1
```

Problem 3

```
class vec3 {
public:
    int* p;
    vec3(int x, int y, int z) : p(new int[3]) {
        p[0] = x; p[1] = y; p[2] = z;
    }
    void print() {
        if (p == nullptr) {
            cout << "This vector's data has been deleted." << endl;
        } else {
            cout << "[" << p[0] << ", " << p[1] << ", " << p[2] << "]" << endl;
        }
    }
    vec3 (vec3& other) : p(other.p)
    {
        other.p = nullptr;
    }
    vec3(vec3&& other) : p(new int[3]) {
        p[0] = other.p[0];
        p[1] = other.p[1];
        p[2] = other.p[2];
    }
    ~vec3() {delete[] p;}
```

```
};

int main() {
    cout << boolalpha;
    vec3 v(1 , 2 , 2);
    int* v_p = v.p;
    vec3 w(v);
    v.print();
    w.print();
    cout << (v_p == v.p) << " " << (w.p == v.p) << " " << (v_p == w.p) << endl;
    vec3&& ref = vec3(1 , 1 , 1);
    int* r_p = ref.p;
    vec3 r(move(ref));
    ref.print();
    r.print();
    cout << (r.p == ref.p) << " " << (r.p == r_p) << " " << (r_p == ref.p) << endl;
}
```

Answer

The first `cout` statement here will print a couple of checks. First it checks if `v_p` is equal to `v.p`. Note that the copy constructor was called to create `w`, but in this implementation, the copy and move constructors have been flipped (in terms of what they are supposed to do). We see in the copy constructor, `other.p = nullptr;`, so `v.p` has been set to `nullptr`. The vector `w` now has control over the data that was in `v`. So `v_p == v.p` will print false. Next `w.p` will not be equal to `v.p` because `w.p` now has the point for the data that used to be in `v`, and `v` has a null pointer. We will get `v_p` is equalt o `w.p` because `v_p` is the original pointer from `v`. The print statements before the first `cout` statement will reflect this as well.

In the second half of the code, an r-value reference `ref` is created, keeping alive the r-value `vec3(1,1,1);`. The move constructor is invoked to create `r` (note that we still have to use `move(ref)`, even though `ref` is an r-value reference, recall that any named value except `this` is automatically an l-value). Printing onf `ref` and `r` will both yield `[1, 1, 1]`. Because the move constructor actually is making a copy, `r.p` will not be equal to `ref.p`, `r.p` will not be equal to `r_p` and `r_p` will be equal to `ref.p`.

```
This vector's data has been deleted
[1, 2, 3]
false false true
[1, 1, 1]
[1, 1, 1]
false false true
```

Writing code

Write code which completes the following tasks.

Problem 1

Write an operator overload to print a vector of integers, as in the following code:

```
int main () {
    vector<int> v = {3, -2, 4, 1};
    cout << v; // should print [3 , -2 , 4 , 1].
    v = {};
    cout << v; // should print [].
}
```

Answer

```
std::ostream& operator<<(ostream& out, const vector<int>& v) {
    out << "[";
    for (int i=0; i < v.size(); ++i) {
         out << v[i] << " ";
    }
    out << "]";
    return out;
}
```

Problem 2

Complete the iterator for the class "Collatz" so that the following code builds and has the designated behavior. Recall that the Collatz sequence of an integer k is the recursive sequence given by $a_1 = k$, and

$$a_{n+1} = \begin{cases} 3a_n + 1, & \text{if n odd} \\ a_n/2, & \text{if n even} \end{cases}$$

In this setting, we will define the sequence to terminate once one of the terms is equal to 1.

```
class Collatz {
private:
    unsigned int k;
    class iterator {
    // TODO
    // Hint : need to define operator ++() ,
    // operator *() , and operator !=() .
    };
public:
    Collatz(unsigned int _k) : k(_k) {}
```

```
    iterator begin() { }
    iterator end() { }
};

int main () {
    // Should print 5 , 16 , 8 , 4 , 2 , 1
    for (unsigned int i : Collatz(5)) {
        cout << i << ", ";
    }
    cout << 1 << endl;
    // Should print 7 , 22 , 11 , 34 , 17 , 52 , 26 , 13 , 40 , 20, 10 , 5 , 16 , 8 , 4 , 2
    for ( unsigned int i : Collatz(7)) {
    cout << i << ", ";
    }
    cout << 1 << endl;
}
```

Answer

This is not the only way to do it, you just need `operator++`, `operator*()`, `operator!=` and `begin()`/`end()` to be defined correctly. I also made everything in the iterator public, which is not the safest option here, but it is fine for a minimal working example.

```
class Collatz {
private:
    unsigned int k;
    class iterator {
    public:
        Collatz* containter;
        unsigned int ai;
        iterator(Collatz* cont) : container(cont), ai(cont->k) {};
        iterator(Collatz* cont, int ai_) container(cont), ai(ai_) {};
        iterator& operator++() {
            if (ai % 2 == 0) {
                ai = ai / 2;
            } else {
                ai = 3*ai + 1;
            }
            return this*;
        }
        int operator*() {
            return ai;
        }
        friend bool operator!=(const iterator& left, const iterator& right);
    };
```

```
public:
    Collatz(unsigned int _k) : k(_k) {}
    iterator begin() {return iterator(this);}
    iterator end() {return iterator(this, 1);}
};

bool operator!=(const Collatz::iterator& left, const Collatz::iterator& right) {
    return (left->container != right->container) || (left->ai != right->ai);
}

int main () {
    // Should print 5 , 16 , 8 , 4 , 2 , 1
    for (unsigned int i : Collatz(5)) {
        cout << i << ", ";
    }
    cout << 1 << endl;
    // Should print 7 , 22 , 11 , 34 , 17 , 52 , 26 , 13 , 40 , 20, 10 , 5 , 16 , 8 , 4 , 2
    for ( unsigned int i : Collatz(7)) {
    cout << i << ", ";
    }
    cout << 1 << endl;
}
```